
Unity DOTS

справочник для первого проекта

Игра «Сборщик»: кубик ездит по полю и собирает монетки на время.
Восемь шагов. Код ты пишешь сам — здесь только то, из чего его собрать.

Как этим пользоваться. В каждом шаге написано, что должно заработать, какие файлы создать и какие конструкции для этого существуют. Готовых решений нет намеренно: концепция ECS укладывается в голову, когда сам решаешь, где лежат данные и кто их меняет. Если застрял на синтаксисе — он в таблицах. Если застрял на ошибке — приложение Б.

Порядок важен. Шаги устроены так, что после каждого игра запускается и что-то делает. Не прыгай вперёд: шаг 5 опирается на то, что в шаге 3 ты уже разобрался с бейкингком.

Чего здесь нет. Джобов, Burst, пулов, физики. Всё это в первом проекте только мешает: оно про производительность, а не про концепцию. Игра из восьми шагов работает на главном потоке и этого достаточно.

Unity 6000.x · com.unity.entities 6.5 · URP

Сигнатуры даны для Entities 1.0 и новее; в версиях до 1.0 они выглядели иначе.

1 Подготовка проекта

Цель: пустая сцена, в которой работает ECS, и открытые окна, показывающие сущности. Без них дальше придётся гадать.

ЧТО СДЕЛАТЬ РУКАМИ

- Новый проект на шаблоне **Universal 3D**.
- Package Manager → + → Add package by name → `com.unity.entities.graphics`. Он подтянет `com.unity.entities` сам. Ставить нужно именно `graphics`: без него сущности существуют, но не рисуются.
- В сцене: правый клик в Hierarchy → **New Sub Scene** → Empty Scene, назови `MainSubScene`.
- Открой **Window** → **Entities** → **Hierarchy** и **Window** → **Entities** → **Systems**. Первое окно показывает, что существует в игре, второе — какие системы работают и в каком порядке.

Главное правило всего проекта

Объект становится сущностью **только если лежит внутри SubScene**. В обычной сцене он выглядит в инспекторе точно так же, скрипт компилируется, ошибок нет — но бейкинг не запускается и сущности не появляется. Когда «система ничего не находит», первым делом смотри, внутри ли SubScene лежит объект.

СТРУКТУРА ПАПОК

Заведи сразу: `Scripts/Components` — структуры данных, `Scripts/Authoring` — мосты из сцены в ECS, `Scripts/Systems` — логика, `Scripts/UI` — обычные `MonoBehaviour`. Разделение не формальность: у этих четырёх видов кода разные правила, и держать их вперемешку больно.

КАК ПРОВЕРИТЬ

Положи в SubScene обычный Cube. В окне Entities Hierarchy должна появиться сущность `Cube`; выдели её — справа видны её компоненты: `LocalTransform`, `LocalToWorld` и прочее. Если сущности нет, SubScene не запеклась: выдели её в Hierarchy и нажми в инспекторе **Open**, затем **Close**.

2 Компоненты данных

Цель: описать, какие данные бывают в игре. Логика ещё нет, только структуры. Для понимания ECS это самый важный шаг.

ЧТО СОЗДАТЬ

В `Scripts/Components`, по файлу на компонент: `Speed`, `PlayerTag`, `CoinTag`, `Score`, `GameTimer`, `SpawnerData`.

ИЗ ЧЕГО СОБИРАТЬ

<pre>struct Speed : IComponentData { public float Value; }</pre>	Компонент данных. Обязательно <code>struct</code> , обязательно публичные поля. Это не класс и не <code>MonoBehaviour</code> : нет методов, нет <code>Start</code> , нет ссылок на сцену.
<pre>struct PlayerTag : IComponentData { }</pre>	Тег. Пустая структура. Нужна, чтобы отличать сущности в запросах — «возьми всех, у кого есть <code>PlayerTag</code> ». Памяти не занимает.
<pre>struct Score : IComponentData { public int Value; }</pre>	Синглтон. Технически обычный компонент; синглтоном его делает то, что в игре он существует ровно на одной сущности. Так хранят общее состояние: счёт, таймер, настройки.

ЧТО МОЖНО КЛАСТЬ ВНУТРИ

<pre>float, int, bool, enum float2, float3, quaternion Entity</pre>	Можно. <code>float3</code> и остальная математика — из <code>Unity.Mathematics</code> , не из <code>UnityEngine</code> . <code>Entity</code> — ссылка на другую сущность, внутри это просто пара чисел.
<pre>string, int[], List<T>, GameObject, Transform, class</pre>	Нельзя. Компонент должен быть blittable — простая память без ссылок. Ошибка появится не при компиляции, а позже, от IL-постпроцессора, и текст у неё невнятный.

Где ломается привычка из ООП

Хочется сделать один компонент `Player` со всеми полями: скорость, счёт, здоровье. Так делать не надо — и не из-за производительности, а потому что тогда каждая система зависит от всех данных сразу, и теряется главное свойство ECS: система видит ровно те поля, которые ей нужны.

Правило на первое время: один компонент — одна осмысленная величина. Позицию **не описывай вообще**, она уже есть (шаг 4).

КАК ПРОВЕРИТЬ

Проверять пока нечего — убедись, что проект компилируется. Если в консоли ругань про blittable, внутри компонента остался класс, массив или строка.

3 Игрок на сцене: authoring и Baker

Цель: кубик в SubScene превращается в сущность, у которой есть `PlayerTag` и `Speed` со значением, выставленным в инспекторе.

Бейкинг — это перевод сцены в сущности, он происходит **в редакторе**, до запуска игры. Authoring-компонент это обычный `MonoBehaviour`: он нужен только чтобы ты мог что-то потрогать в инспекторе. В игру он не попадает — в игру попадает то, что Baker из него сделал.

ЧТО СОЗДАТЬ

`Scripts/Authoring/PlayerAuthoring.cs`, повесить на Cube внутри SubScene.

КАРКАС

```
public class PlayerAuthoring : MonoBehaviour
{
    public float Speed;          // видно в инспекторе

    class PlayerBaker : Baker<PlayerAuthoring>
    {
        public override void Bake(PlayerAuthoring authoring)
        {
            // здесь получить сущность и добавить ей компоненты
        }
    }
}
```

ИЗ ЧЕГО СОБИРАТЬ

<code>GetEntity(TransformUsageFlags flags)</code>	Сущность этого объекта. Вызывается внутри <code>Bake</code> . Флаг говорит, какие трансформные компоненты нужны: <code>Dynamic</code> — объект будет двигаться, <code>Renderable</code> — только рисуется и стоит на месте, <code>None</code> — чистые данные без позиции.
<code>AddComponent(entity, new Speed { ... })</code>	Добавить компонент со значением. Значения берутся из полей <code>authoring</code> .
<code>AddComponent<PlayerTag>(entity)</code>	Добавить тег. Форма для пустых структур, значение передавать нечего.
<code>GetEntity(authoring.Prefab, flags)</code>	Сущность другого объекта — понадобится в шаге 5 для префаба. Возвращает <code>Entity</code> , который можно положить в компонент.

Две ловушки, обе тихие

Забыл `AddComponent` — ошибки не будет никакой. Система просто не найдёт ни одной подходящей сущности и будет молча работать впустую. Это самая частая причина «ничего не происходит, но и не падает».

Неверный `TransformUsageFlags` — если поставить `Renderable` объекту, который должен двигаться, у сущности не будет `LocalTransform`, и запрос из шага 4 её не увидит. Для игрока нужен `Dynamic`.

КАК ПРОВЕРИТЬ

Entities Hierarchy → выдели сущность кубика. В списке её компонентов должны быть `PlayerTag` и `Speed`, причём у `Speed` видно значение из инспектора. Поменяй значение в инспекторе — число в окне сущностей должно измениться сразу, без запуска игры. Это и есть бейкинг.

4 Движение по WASD

Цель: кубик едет по нажатию клавиш, упираясь в границы поля. Первая система.

Система — это `struct`, который Unity находит сам и вызывает каждый кадр. Регистрировать его нигде не нужно, достаточно объявить. Слово `partial` обязательно: генератор кода дописывает в систему свою половину, и без `partial` будет ошибка компиляции.

КАРКАС

```
public partial struct PlayerMoveSystem : ISystem
{
    public void OnCreate(ref SystemState state) { } // один раз, до игры
    public void OnUpdate(ref SystemState state) { } // каждый кадр
    public void OnDestroy(ref SystemState state) { } // при выключении
}
```

Все три метода необязательны: у `ISystem` есть пустые реализации по умолчанию, лишние можно не писать.

ПОЗИЦИЯ, ПОВОРОТ, МАСШТАБ

<code>LocalTransform</code>	Трансформ сущности. Отдельный компонент писать не надо, он появляется сам при флаге <code>Dynamic</code> . Поля: <code>Position</code> (<code>float3</code>), <code>Rotation</code> (<code>quaternion</code>), <code>Scale</code> (<code>float</code> — один, а не три).
<code>LocalTransform.FromPosition(float3)</code>	Создать трансформ. Есть также <code>FromPositionRotation</code> , <code>FromPositionRotationScale</code> , <code>LocalTransform.Identity</code> .

ПЕРЕБОР СУЩНОСТЕЙ

<code>foreach (var (a, b) in SystemAPI .Query<RefRW<LocalTransform>, RefRO<Speed>>())</code>	Основной цикл ECS. Перебирает все сущности, у которых есть все перечисленные компоненты. Остальные не попадут.
<code>RefRW<T> → .ValueRW RefRO<T> → .ValueRO</code>	Чтение и запись. <code>RW</code> — собираешься менять, <code>RO</code> — только читать. Ставь <code>RO</code> везде, где можно: это не стиль, а информация для планировщика.
<code>.WithAll<PlayerTag>()</code>	Фильтр по тегу. Дописывается после <code>Query</code> . Нужен, когда сам компонент в цикле не используется, а отличить сущности надо. Есть ещё <code>WithNone<T>()</code> , <code>WithAny<T>()</code> .
<code>.WithEntityAccess()</code>	Добавляет саму сущность последним элементом кортежа. Нужно, когда её собираешься удалить или изменить — см. шаг 6.
<code>SystemAPI.Time.DeltaTime</code>	Время кадра. Не <code>Time.deltaTime</code> из <code>UnityEngine</code> . Умножай на него любое движение, иначе скорость будет зависеть от частоты кадров.

ВВОД И МАТЕМАТИКА

```
Input.GetAxis("Horizontal")
Input.GetAxis("Vertical")
```

Оси **WASD** и **стрелок**, от -1 до 1 . Это обычный UnityEngine, и работает он только на главном потоке — то есть прямо в `OnUpdate`. Внутрь джобы его вынести нельзя.

```
math.clamp(v, min, max)
math.normalize(v)
math.lengthsq(v)
```

Математика из `Unity.Mathematics`. Для границ поля нужен `clamp` по X и Z.

Диагональ едет быстрее

Если складывать ввод по осям напрямую, при двух нажатых клавишах длина вектора будет 1.41 вместо 1, и по диагонали кубик поедет на 41% быстрее. Нормализуй вектор ввода — но только когда он не нулевой:

`math.normalize` от нуля даёт NaN, а NaN в позиции убивает сущность навсегда.

КАК ПРОВЕРИТЬ

Кубик едет по WASD с той скоростью, что стоит в инспекторе, и не уезжает за край поля. Зажми две клавиши: по диагонали скорость должна быть такой же, как по прямой.

5 Спавн монеток

Цель: раз в N секунд в случайном месте поля появляется монетка. Здесь впервые создаются сущности во время игры.

Сущность нельзя просто создать посреди перебора: создание и удаление меняют структуру памяти и ломают цикл, внутри которого ты находишься. Поэтому команды складывают в буфер и проигрывают после.

ЧТО СОЗДАТЬ

Префаб монетки (Cube поменьше, свой материал) + `CoinAuthoring` с `CoinTag`. Пустой объект `Spawner` в `SubScene` + `SpawnerAuthoring` с полем `public GameObject CoinPrefab`. Система `CoinSpawnSystem`.

Префаб тоже надо запечь

Положить `GameObject` в компонент нельзя. В Baker спавнера вызови `GetEntity(authoring.CoinPrefab, TransformUsageFlags.Dynamic)` — вернётся `Entity`, вот его и храни в `SpawnerData.Prefab`. Сам префаб при этом не обязан лежать в сцене.

СОЗДАНИЕ СУЩНОСТЕЙ

<pre>state.EntityManager .Instantiate(prefabEntity)</pre>	Копия префаба, немедленно. Возвращает <code>Entity</code> , которому можно сразу задать позицию. Простой путь, годится, пока создаёшь по одной сущности вне перебора.
<pre>var ecb = new EntityCommandBuffer(Allocator.Temp); ecb.Playback(state.EntityManager); ecb.Dispose();</pre>	Буфер команд. Собирает действия и выполняет их все сразу в момент <code>Playback</code> . Так создают и удаляют сущности, находясь внутри перебора.
<pre>ecb.Instantiate(prefab) ecb.DestroyEntity(entity) ecb.SetComponent(e, value) ecb.AddComponent(e, value)</pre>	Команды буфера. Выполняются не сразу, а при <code>Playback</code> . <code>Instantiate</code> возвращает «обещание» сущности — ему можно задавать компоненты, хотя её ещё нет.
<pre>state.EntityManager .SetComponentData(e, value)</pre>	Задать компонент немедленно. Парный <code>GetComponentData<T>(e)</code> . Внутри перебора использовать нельзя — только до или после.

ТАЙМЕР И СЛУЧАЙНОСТЬ

<pre>Unity.Mathematics.Random Random.CreateFromIndex(uint seed) rnd.NextFloat(min, max) rnd.NextInt(min, max)</pre>	Случайные числа для ECS. Это структура с состоянием: её надо хранить в компоненте и возвращать обратно после использования, иначе каждый кадр выпадет одно и то же число. <code>UnityEngine.Random</code> в ECS не используют.
---	---

поле `float Timer` в компоненте

Как отсчитывать время. Своей памяти у системы нет — точнее, есть поля структуры, но хранить состояние игры в них неправильно. Прибавляй `DeltaTime` к полю компонента, при достижении порога спавни и вычитай порог.

Состояние системы живёт в компонентах

Соблазн объявить `private float _timer` полем системы. Так делать не надо: система концептуально не владеет данными, она их только обрабатывает. Таймер — это данные игры, его место в `SpawnerData`. То же про `Random`.

КАК ПРОВЕРИТЬ

Монетки появляются в разных местах с нужным интервалом, и число сущностей в `Entities Hierarchy` растёт. Если все появляются в одной точке — не вернул состояние `Random` обратно в компонент.

6 Подбор монеток и счёт

Цель: когда игрок подъезжает к монетке, она исчезает, счёт растёт на единицу.

Здесь появляется задача, непривычная после ООП: нужно сравнить игрока с каждой монеткой, то есть совместить два разных перебора. Прямого способа нет и это нормально — позиция игрока берётся до цикла, одним запросом.

ЧТО СОЗДАТЬ

Система `CoinPickupSystem`. Сущность-синглтон со `Score`: либо отдельный пустой объект в `SubScene` с `authoring`-компонентом, либо создать её в `OnCreate` системы (см. ниже).

СИНГЛТОНЫ

<code>SystemAPI.GetSingleton<Score>()</code>	Прочитать синглтон. Возвращает копию — изменения в ней никуда не попадут. Бросает исключение, если сущностей не ровно одна.
<code>SystemAPI.GetSingletonRW<Score>().ValueRW.Value++</code>	Изменить синглтон. Именно это нужно для счёта. Самая частая ошибка новичка — менять результат <code>GetSingleton</code> и удивляться, что ничего не меняется.
<code>SystemAPI.TryGetSingleton<T>(out T v)</code>	Безопасный вариант — <code>false</code> вместо исключения, если синглтона нет.
<code>SystemAPI.GetSingletonEntity<T>()</code>	Сама сущность синглтона, а не его значение.
<code>state.EntityManager.CreateEntity() + AddComponent<Score>(e)</code>	Создать синглтон кодом , в <code>OnCreate</code> . Удобно, когда в инспекторе настраивать нечего: не нужен ни объект в сцене, ни <code>authoring</code> .

ПОИСК СОСЕДЕЙ И УДАЛЕНИЕ

<code>math.distancesq(a, b)</code>	Квадрат расстояния. Сравнивай с квадратом радиуса: <code>distancesq < r * r</code> . Корень не считается, а для сравнения он и не нужен.
<code>.WithEntityAccess()</code>	Нужен здесь обязательно — чтобы получить <code>Entity</code> монетки и передать его на удаление.
<code>ecb.DestroyEntity(entity)</code>	Удалить сущность после перебора. Напрямую <code>EntityManager.DestroyEntity</code> внутри <code>foreach</code> вызывать нельзя.

Как получить позицию игрока внутри цикла по монеткам

Не вкладывая один `SystemAPI.Query` в другой. Правильно: до цикла одной строкой достать трансформ игрока — через запрос с `WithAll<PlayerTag>()` или через синглтон, если игрок один, — положить в локальную переменную `float3`, и дальше перебирать монетки, сравнивая с ней.

КАК ПРОВЕРИТЬ

Монетка исчезает при наезде. Значение `Score` в Entities Hierarchy растёт — выдели сущность со счётом и смотри на него во время игры. На экране его ещё нет, это следующий шаг.

7 Счёт и таймер на экране

Цель: счёт и остаток времени видны на экране. Здесь ECS соединяется с обычным Unity.

UI в DOTS не живёт — Canvas, Text и кнопки остаются обычными MonoBehaviour. Значит нужен мост: скрипт, который из Update заглядывает в мир сущностей и переписывает текст.

ЧТО СОЗДАТЬ

Canvas с двумя TextMeshProUGUI. Scripts/UI/GameHud.cs — обычный MonoBehaviour на Canvas. Систему GameTimerSystem, которая вычитает DeltaTime из GameTimer.

ДОСТУП К ECS ИЗ МОНОБЕHAVIOUR

World.DefaultGameObjectInjectionWorld .EntityManager	Вход в мир сущностей из обычного кода. Мир один, создаётся автоматически.
entityManager.CreateEntityQuery(ComponentType.ReadOnly<Score>())	Запрос , который потом можно спрашивать каждый кадр. Создавать его надо один раз в Start и хранить в поле.
query.GetSingleton<Score>()	Прочитать значение . У запроса свои методы, похожие на SystemAPI.
query.CalculateEntityCount()	Сколько подходящих сущностей . Так выводят «монеток на поле: N» без всякого счётчика. Полезно и как проверка перед чтением синглтона.
query.CompleteDependency()	Дождаться систем , которые пишут эти компоненты. В простом проекте, где всё на главном потоке, не нужно — но привыкай ставить: как только появятся джобы, без этой строки чтение падает каждый кадр.
query.ToComponentDataArray<T>(Allocator.Temp)	Все значения массивом , если нужно не одно. Массив надо освободить: .Dispose() или using.

Два правила моста

Запрос создаётся в Start, не в Update. Создание запроса — операция недешёвая, а делать её 60 раз в секунду незачем.

Порядок загрузки не гарантирован. MonoBehaviour может стартовать раньше, чем запеклась SubScene, и синглтона в первый кадр ещё не будет. Проверь CalculateEntityCount() == 1 перед чтением, иначе поймает исключение на старте.

КАК ПРОВЕРИТЬ

Счёт на экране совпадает с тем, что показывает Entities Hierarchy. Таймер убывает. Запусти и останови игру несколько раз подряд — исключений в первом кадре быть не должно.

8 Конец игры и перезапуск

Цель: время вышло — игра останавливается, появляется окно с результатом и кнопкой. Кнопка возвращает всё в начальное состояние.

Нужен флаг, который видят и системы, и UI. Это синглтон с состоянием — тот же приём, что со счётом, но теперь его читают почти все.

ЧТО СОЗДАТЬ

Компонент `GameState` с полями `bool IsOver`, `bool RestartRequested`. Систему `GameRestartSystem`. Панель `Game Over` с кнопкой, скрипт на ней пишет `RestartRequested = true`.

ОСТАНОВКА СИСТЕМ

<pre>if (SystemAPI.GetSingleton<GameState>() .IsOver) return;</pre>	Самый простой способ. Первая строка в <code>OnUpdate</code> каждой системы, которую надо приглушить: движение, спавн, подбор, таймер. Понятно и отлаживается легко.
<pre>state.Enabled = false;</pre>	Выключить систему насовсем. Годится для систем, которые делают работу один раз в <code>OnCreate</code> . Для игровой логики не подходит: включить обратно сложнее, чем кажется.
<pre>state.RequireForUpdate<T>()</pre>	Не вызывать <code>OnUpdate</code>, пока нет такого компонента. Ставится в <code>OnCreate</code> . Удобно, но опасно: если компонент пропал, система молча не работает и в консоли ничего нет.

СБРОС СОСТОЯНИЯ

<pre>query.ToEntityArray(Allocator.Temp) → em.DestroyEntity(array)</pre>	Удалить все монетки. Сначала собрать массив сущностей, потом удалить — внутри перебора удалять нельзя.
<pre>em.DestroyEntity(EntityQuery)</pre>	Есть и такая перегрузка, но она требует, чтобы в запрос попали все связанные дочерние сущности, иначе бросает <code>ArgumentException</code> . С составными префабами проще через массив.
<pre>em.SetComponentData(e, value)</pre>	Вернуть значения. Счёт в ноль, таймер на полную, позицию игрока в центр, <code>IsOver = false</code> .

Что тут обычно забывают

Сбросить **таймер спавнера**. Он остался от прошлой партии, и первая монетка либо появится мгновенно, либо через полный интервал вместо нужного. Правило общее: при перезапуске возвращай в исходное *всё* изменяемое состояние, а не только то, что видно на экране.

КАК ПРОВЕРИТЬ

Время вышло → кубик не едет, монетки не появляются, видно окно с результатом. Кнопка → поле чистое, счёт ноль, таймер полный, игрок в центре. Пройди три партии подряд, не выходя из Play: вторая и третья должны быть идентичны первой.

КУДА ИДТИ ДАЛЬШЕ

Когда это заработает, в игре есть все базовые приёмы ECS: данные в компонентах, логика в системах, бейкинг из сцены, отложенные команды, синглтоны, мост в UI. Следующие темы по порядку полезности: `IJobEntity` и `[BurstCompile]` (та же логика, но в несколько потоков), `IEnableableComponent` и пулы (перестать создавать и удалять сущности каждую секунду), группы систем и явный порядок через `[UpdateBefore]`, буферы (`IBufferElementData`) для передачи событий в UI и звук.

Приложение А. Хочу сделать X

Обратный указатель: по задаче — к нужной конструкции.

ДАННЫЕ	
Хранить число у сущности	<code>struct X : IComponentData</code> с полем
Отличать один вид сущностей от другого	Пустой <code>tag</code> + <code>WithAll<Tag>()</code>
Хранить одно значение на всю игру	Компонент на одной сущности + <code>GetSingletonRW</code>
Хранить список переменной длины	<code>IBufferElementData</code> вместо массива в компоненте
Настраивать значение в инспекторе	Поле в <code>authoring</code> + <code>AddComponent</code> в <code>Baker</code>
Сослаться на другую сущность	Поле типа <code>Entity</code>
ПЕРЕБОР И ПОИСК	
Пройти по всем сущностям с А и В	<code>SystemAPI.Query<RefRW<A>, RefRO>()</code>
Исключить сущности с компонентом С	<code>.WithNone<C>()</code>
Получить саму сущность в цикле	<code>.WithEntityAccess()</code>
Сколько всего таких сущностей	<code>query.CalculateEntityCount()</code>
Найти ближайшую цель	Цикл с <code>math.distancesq</code> и минимумом
Прочитать компонент чужой сущности	<code>SystemAPI.GetComponent<T>(entity)</code>
Проверить, есть ли компонент	<code>SystemAPI.HasComponent<T>(entity)</code>
ИЗМЕНЕНИЯ	
Создать сущность вне перебора	<code>EntityManager.Instantiate(prefab)</code>
Создать или удалить внутри перебора	<code>EntityCommandBuffer</code> + <code>Playback</code>
Добавить компонент на ходу	<code>ecb.AddComponent(e, value)</code>
Включить и выключить сущность	<code>IEnableableComponent</code> + <code>SetComponentEnabled</code>
ВРЕМЯ, ВВОД, МАТЕМАТИКА	
Время кадра	<code>SystemAPI.Time.DeltaTime</code>
Отсчитать N секунд	<code>float Timer</code> в компоненте, не в системе
Клавиатура	<code>Input.GetAxis</code> в <code>OnUpdate</code> , вне джоб
Случайное число	<code>Unity.Mathematics.Random</code> в компоненте
Повернуть к цели	<code>quaternion.LookRotationSafe(dir, math.up())</code>

Ограничить по полю	<code>math.clamp</code> по каждой оси
--------------------	---------------------------------------

СВЯЗЬ С ОБЫЧНЫМ UNITY

Прочитать данные ECS из MonoBehaviour	Запрос в <code>Start</code> , чтение в <code>Update</code>
Нажать кнопку и повлиять на ECS	Записать флаг в синглтон через <code>EntityManager</code>
Проиграть звук по событию из ECS	Буфер событий + MonoBehaviour-читатель, он же чистит буфер

ПОРЯДОК РАБОТЫ СИСТЕМ

Задать группу	<code>[UpdateInGroup(typeof(SimulationSystemGroup))]</code>
Раньше или позже другой системы	<code>[UpdateBefore(typeof(X))]</code> , <code>[UpdateAfter]</code>
Сделать что-то один раз на старте	<code>OnCreate</code>

Приложение Б. Ошибки и что они значат

Главная часть справочника. Половина времени новичка уходит на расшифровку этих сообщений.

НЕ КОМПИЛИРУЕТСЯ

<code>...must be declared partial</code>	У системы нет слова <code>partial</code> . Генератор кода дописывает в неё свою часть, и без этого слова не может.
<code>field of type ... is neither primitive nor blittable</code>	В <code>IComponentData</code> лежит класс, массив, строка или <code>List</code> . Убрать; для переменной длины — буфер.
<code>The type or namespace 'float3' could not be found</code>	Нет <code>using Unity.Mathematics;</code> . Остальные обычные: <code>Unity.Entities</code> , <code>Unity.Transforms</code> , <code>Unity.Collections</code> .

ПАДАЕТ В ИГРЕ

<code>GetSingleton... requires exactly one entity but found 0</code>	Сущности нет вообще. Причины по частоте: <code>authoring</code> лежит вне <code>SubScene</code> ; забыл <code>AddComponent</code> в <code>Baker</code> ; читаешь в первом кадре, когда <code>SubScene</code> ещё не запеклась.
<code>...but found 2</code>	Синглтон запёкся дважды — обычно <code>authoring</code> висит и на объекте, и на префабе, либо объект продублирован в сцене.
<code>Attempt to add duplicate component ...</code>	У сущности может быть только один компонент каждого типа. Нужно несколько — нужно несколько сущностей (<code>Baker.CreateAdditionalEntity</code>).
<code>The entity does not exist / has been destroyed</code>	Держишь <code>Entity</code> удалённой сущности. Проверь <code>EntityManager.Exists(e)</code> , либо не удаляй дважды: при двух попаданиях в один кадр <code>DestroyEntity</code> вызовется два раза.
<code>InvalidOperationException: ...was invalidated by a structural change</code>	Создал, удалил или добавил компонент прямо посреди <code>foreach</code> . Все такие действия — через <code>EntityCommandBuffer</code> .
<code>The system ... writes ComponentTypeHandle<T> ...you must call CompleteDependency</code>	Читаешь компонент из <code>MonoBehaviour</code> , пока его пишет джоба. <code>query.CompleteDependency()</code> перед чтением. Коварно тем, что исключение летит до присваивания: на экране просто ничего не обновляется.
<code>ObjectDisposedException / The NativeArray has been deallocated</code>	Либо забыл <code>Dispose</code> и используешь освобождённое, либо объявил контейнер через <code>new()</code> без аллокатора — тогда внутри нет памяти вообще.

НЕ ПАДАЕТ, НО И НЕ РАБОТАЕТ

Система не делает ничего, ошибок нет	Запрос не находит сущностей. Смотри в Entities Hierarchy, какие компоненты реально есть: чаще всего забыт <code>AddComponent</code> в Baker или сущности не хватает <code>LocalTransform</code> из-за флага <code>Renderable</code> вместо <code>Dynamic</code> .
Сущности нет в Entities Hierarchy	Объект вне SubScene, или SubScene не перепеклась. Выдели её, Open → Close .
Сущность есть, но её не видно в игре	Не установлен <code>com.unity.entities.graphics</code> ; или у объекта нет <code>MeshRenderer</code> ; или флаг <code>None</code> вместо <code>Renderable</code> ; или сущность уехала в NaN.
Изменил синглтон — значение не поменялось	<code>GetSingleton</code> возвращает копию. Для записи — <code>GetSingletonRW<T>().ValueRW</code> .
Объект улетает в бесконечность или исчезает навсегда	NaN в позиции. Обычно <code>math.normalize</code> от нулевого вектора. Проверь длину до нормализации.
Движение зависит от частоты кадров	Не умножил на <code>SystemAPI.Time.DeltaTime</code> .
Событие срабатывает каждый кадр вместо одного раза	Читатель буфера не вызвал <code>Clear()</code> после обработки.
Правки <code>authoring</code> не влияют на игру	SubScene открыта на редактирование (Open) — тогда работают <code>GameObject</code> 'ы, а не сущности. Закрой её.

Приложение В. Чем смотреть, что происходит

В ECS нельзя просто нажать на объект в Hierarchy и увидеть состояние. Инструменты другие, и без них отладка превращается в угадывание.

Window → Entities → Hierarchy	Главное окно. Все сущности мира и их компоненты со значениями, живьём во время игры. Первое место, куда смотреть, когда «не работает». Есть поиск по имени компонента.
Window → Entities → Systems	Список систем в порядке выполнения, с галочками включения и временем работы. Видно, попала ли твоя система в нужную группу и вызывается ли вообще.
Window → Entities → Components	Все типы компонентов в проекте и сколько сущностей имеет каждый. Быстрый ответ на «запеклось ли оно вообще».
Window → Entities → Archetypes	Наборы компонентов и память под них. Для первого проекта любопытно, но не обязательно.
Инспектор SubScene: Open / Close	Переключение режима. Open — правишь объекты, сущностей нет. Close — объекты запекаются в сущности. Многие «необъяснимые» проблемы сводятся к забытому Open.
Debug.Log в OnUpdate	Работает, пока система без <code>[BurstCompile]</code> и логика не в джобе. Внутри Burst-кода логировать нельзя — это одна из причин, почему Burst в первом проекте только мешает.

Порядок разбора любой поломки

- Есть ли ошибка в консоли. Если есть — приложение Б, не угадывай.
- Существует ли сущность. Entities Hierarchy.
- Те ли у неё компоненты и те ли значения. Там же.
- Вызывается ли система. Entities → Systems, или `Debug.Log`.
- Находит ли запрос хоть что-то. `CalculateEntityCount()` в лог.

Четыре из пяти поломок новичка находятся на втором или третьем пункте, и оба проверяются глазами за десять секунд.